



RecNMP: Accelerating Personalized Recommendation with Near-Memory Processing

Liu Ke*, Udit Gupta[†], Benjamin Youngjae Cho[§],

David Brooks[†], Vikas Chandra, Utku Diril, Amin Firoozshahian, Kim Hazelwood, Bill Jia, Hsien-Hsin S. Lee, Meng Li, Bert Maher, Dheevatsa Mudigere, Maxim Naumov, Martin Schatz, Mikhail Smelyanskiy, Xiaodong Wang,

Brandon Reagen, Carole-Jean Wu, Mark Hempstead[‡], Xuan Zhang*

Facebook, Inc.

Abstract—Personalized recommendation systems leverage deep learning models and account for the majority of data center AI cycles. Their performance is dominated by memory-bound sparse embedding operations with unique irregular memory access patterns that pose a fundamental challenge to accelerate. This paper proposes a lightweight, commodity DRAM compliant, near-memory processing solution to accelerate personalized recommendation inference. The in-depth characterization of production-grade recommendation models shows that embedding operations with high model-, operator- and data-level parallelism lead to memory bandwidth saturation, limiting recommendation inference performance. We propose *RecNMP* which provides a scalable solution to improve system throughput, supporting a broad range of sparse embedding models. *RecNMP* is specifically tailored to production environments with heavy co-location of operators on a single server. Several hardware/software co-optimization techniques such as memory-side caching, table-aware packet scheduling, and hot entry profiling are studied, providing up to 9.8× memory latency speedup over a highly-optimized baseline. Overall, *RecNMP* offers 4.2× throughput improvement and 45.8% memory energy savings.

I. INTRODUCTION

Personalized recommendation is a fundamental building block of many internet services used by search engines, social networks, online retail, and content streaming [4], [19], [20], [63]. Today’s personalized recommendation systems leverage deep learning to maximize accuracy and deliver the best user experience [21], [25], [29], [44], [49]. The underlying deep learning models now consume the majority of the datacenter cycles spent on AI. For example, recent analysis reveals that the top recommendation models collectively contribute to more than 79% of all AI inference cycles across Facebook’s production datacenters [75].

Despite the large computational demand and production impact, relatively little research has been conducted to optimize deep learning (DL)-based recommendation. Most research efforts within the architecture community have focused on accelerating the compute-intensive, highly-regular computational

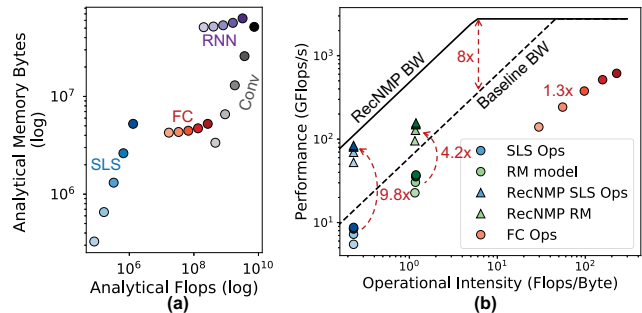


Fig. 1. (a) Compute and memory footprint of common deep learning operators, sweeping batch size; (b) Roofline lifting effect and the operator-level (FC, SLS) and end-to-end model (RM) speedup enabled by *RecNMP*.

patterns found in fully-connected (FC), convolution (CNN), and recurrent (RNN) neural networks [3], [6]–[8], [11], [12], [15], [17], [18], [24], [33], [36], [40], [47], [53]–[55], [57], [60], [68], [71]–[74], [76], [79], [82]. Unlike CNNs and RNNs, recommendation models exhibit low compute-intensity and little to no regularity. Existing acceleration techniques either do not apply or offer small improvements at best, as they tend to exploit regular reusable dataflow patterns and assume high spatial locality, which are not the main performance bottleneck in recommendation models [75]. Given the volume of personalized inferences and their rapid growth rate occurring in the data center, an analogous effort to improve performance of these models would have substantial impact.

To suggest personalized contents to individual users, recommendation models are generally structured to take advantage of both continuous (dense) and categorical (sparse) features. The latter are captured by large embedding tables with sparse lookup and pooling operations. These embedding operations dominate the run-time of recommendation models and are markedly distinct from other layer types.

A quantitative comparison of the raw compute and memory access requirements is shown in Figure 1(a). Sparse embedding operations, represented by SparseLengthsSum (SLS), consist of a small sparse lookup into a large embedding table followed by a reduction of the embedding entries (i.e., pooling). They present two unique challenges: First, while the

*Washington University in St. Louis, work done while at Facebook.

[†]Harvard University, work done while at Facebook.

[§]University of Texas at Austin, work done while at Facebook.

[‡]Tufts University, work done while at Facebook.

sparse lookup working set is comparatively small (MBs), the irregular nature of the table indices exhibits poor predictability, rendering typical prefetching and dataflow optimization techniques ineffective. Second, the embedding tables are on the order of tens to hundreds of GBs, overwhelming on-chip memory resources. Furthermore, the circular points in Figure 1(b) show the operational intensity of SLS is orders of magnitude less than FC layers. Low intensity limits the potential of custom hardware including the specialized datapaths and on-chip memories used in CNN/RNN accelerators. The result is a fundamental memory bottleneck that cannot be overcome with standard caching (e.g., tiling [34]), algorithmic (e.g., input batching), or hardware acceleration techniques.

This paper proposes *RecNMP*—a near-memory processing solution to accelerate the embedding operations for DL-based recommendation. *RecNMP* is a lightweight DIMM-based system built on top of existing standard DRAM technology. We focus on DIMM-based near-memory processing [5], [22], [81] instead of resorting to specialized 2.5D/3D integration processes (e.g. HBM) [17], [38], [45]. The DIMM form factor with commodity DDR4 devices can support the 100GB+ capacities necessary for production-scale recommendation models with low cost. By eliminating the off-chip memory bottleneck and exposing higher internal bandwidth we find that *RecNMP* provides significant opportunity to improve performance and efficiency by lifting the roofline by $8\times$ for the bandwidth-constrained region (Figure 1(b)), enabling optimization opportunity not feasible with existing systems.

We have performed a detailed characterization of recommendation models using open-source, production-scale DLRM benchmark [49], [75] as a case study. This analysis quantifies the potential benefits of near-memory processing in accelerating recommendation models and builds the intuition for co-designing the NMP hardware with the algorithmic properties of recommendation. Specifically, it highlights the opportunity for the *RecNMP* architecture in which bandwidth-intensive embedding table operations are performed in the memory and compute-intensive FC operators are performed on the CPU (or potentially on an accelerator).

The proposed *RecNMP* design exploits DIMM- and rank-level parallelism in DRAM memory systems. *RecNMP* performs local lookup and pooling functions near memory, supporting a range of sparse embedding inference operators, which produce the general Gather-Reduce execution pattern. In contrast to a general-purpose NMP architecture, we make a judicious design choice to implement selected lightweight functional units with small memory-side caches to limit the area overhead and power consumption. We combine this lightweight hardware with software optimizations including table-aware packet scheduling and hot entry profiling. Compared to previous work whose performance evaluation is solely based on randomly-generated embedding accesses [81], our characterization and experimental methodology is modeled after representative production configurations and is evaluated using real production embedding table traces. Overall, *RecNMP* leads to significant embedding access latency reduction

($9.8\times$) and improves end-to-end recommendation inference performance ($4.2\times$) as illustrated in Figure 1(b). Our work makes the following research contributions:

- Our in-depth workload characterization shows that production recommendation models are constrained by memory bandwidth. Our locality analysis using production embedding table traces reveals distinctive spatial and temporal reuse patterns and motivates a custom-designed NMP approach for recommendation acceleration.
- We propose *RecNMP*, a lightweight DDR4-compatible near-memory processing architecture. *RecNMP* accelerates the execution of a broad class of recommendation models and provides $9.8\times$ memory latency speedup and 45.9% memory energy savings. Overall, *RecNMP* achieves $4.2\times$ end-to-end throughput improvement.
- We examine *hardware-software co-optimization* techniques (memory-side caching, table-aware packet scheduling, and hot entry profiling) to enhance *RecNMP* performance, and customized NMP instruction with $8\times$ DRAM command/address bandwidth expansion.
- A *production-aware evaluation framework* is developed to take into account common data-center practices and representative production configuration, such as model co-location and load balancing.

II. CHARACTERIZING DEEP LEARNING PERSONALIZED RECOMMENDATION MODELS

This section describes the general architecture of DL-based recommendation models with prominent sparse embedding features and their performance bottlenecks. As a case study, we conduct a thorough characterization of the recently-released Deep Learning Recommendation Model (DLRM) benchmark [49]. The characterization—latency breakdown, roofline analysis, bandwidth analysis, and memory locality—illustrates the unique memory requirements and access behavior of production-scale recommendation models and justifies the proposed near-memory accelerator architecture.

A. Overview of Personalized Recommendation Models

Personalized recommendation is the task of recommending content to users based on their preferences and previous interactions. For instance, video ranking (e.g., Netflix, YouTube), a small number of videos, out of potentially millions, must be recommended to each user. Thus, delivering accurate recommendations in a timely and efficient manner is important.

Most modern recommendation models have an extremely large feature set to capture a range of user behavior and preferences. These features are typically separated out into dense and sparse features. While dense features (i.e., vectors, matrices) are processed by typical DNN layers (i.e., FC, CNN, RNN), sparse features are processed by indexing large embedding tables. A general model architecture of DL-based recommendation systems is captured in Figure 2. A few examples are listed with their specific model parameters [49], [52], [63] in Figure 2(b). Similar mixture of dense and

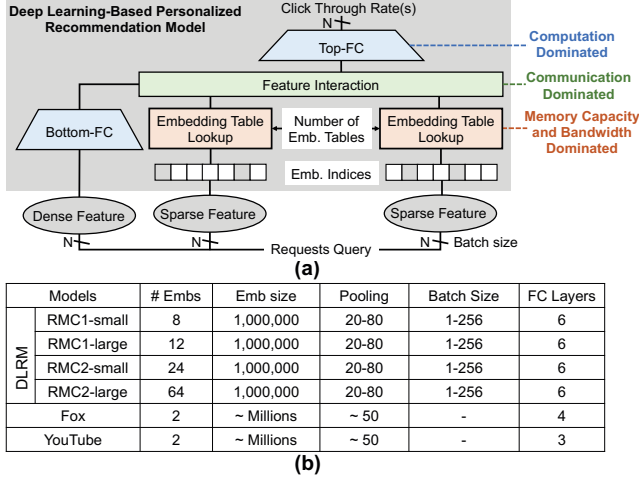


Fig. 2. (a) Simplified model-architecture reflecting production-scale recommendation models; (b) Parameters of representative recommendation models.

sparse features are broadly observable across many alternative recommendation models [1], [25], [49], [52], [63], [83].

Embedding table lookup and pooling operations provide an abstract representation of sparse features learned during training and are central to DL-based recommendation models. Embedding tables are organized as a set of potentially millions of vectors. Generally, embedding table operations exhibit Gather-Reduce pattern; the specific element-wise reduction operation varies between models. For example, Caffe [2] comprises a family of embedding operations, prefixed by *SparseLengths* (i.e., *SparseLengthsWeightedSum8BitsRowwise*), that perform a similar Gather-Reduce embedding operation with quantized, weighted summation. The SLS operator primitive is widely employed by other production-scale recommendation applications (e.g. YouTube [63] and Fox [52]). Our work aims to alleviate this performance bottleneck and improve system throughput by devising a novel NMP solution to offload the SLS-family embedding operations thus covering a broad class of recommendation systems.

B. A Case Study—Facebook’s DLRM Benchmark

To demonstrate the advantages of near-memory processing for at-scale personalized recommendation models, we study Facebook’s deep learning recommendation models (DLRMs) [49]. Dense features are initially processed by the BottomFC operators, while sparse input features are processed through the embedding table lookups. The output of these operators are combined and processed by TopFC producing a prediction of click-through-rate of the user-item pair.

This paper focuses on performance acceleration strategies for four recommendation models representing two canonical classes of the models, RMC1 and RMC2 [75]. These two recommendation model classes consume significant machine learning execution cycles at Facebook’s production datacenter, with RMC1 over 29%, RMC2 over 31%. Parameters to configure are shown in Figure 2(b). The notable distinguishing factor across these configurations is the number of the embedding

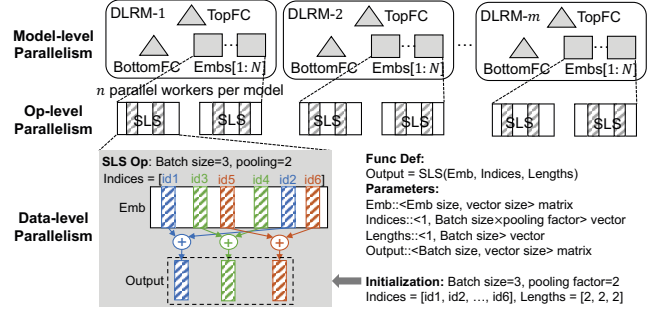


Fig. 3. Model-, operator- and data-level parallelism in production system.

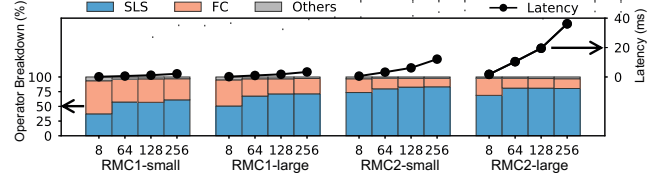


Fig. 4. Inference latency and breakdown across models (RMC1-small, RMC1-large, RMC2-small, RMC2-large) with varying batch sizes (8, 64, 128, 256).

tables. RMC1 is a comparatively smaller model with few embedding tables; RMC2 has tens of embedding tables.

Recommendation systems employ three levels of parallelism, shown in Figure 3, to achieve high throughput under strict latency constraints [75]. Model-level parallelism grows by increasing the number of concurrent model inference (m) on a single machine, operator-level parallelism adds parallel threads (n) per model and data-level parallelism is scaled by increasing batch size. An SLS operator performs a batch of pooling operations; one pooling operation performs the summation for a set of vectors. The inputs to SLS, for one batch of embedding lookups, include an indices vector containing sparse-IDs, and optionally a weight vector.

C. Operator Bottleneck Study

We observe that *the SLS-family of operators is the largest contributor to latency* in recommendation models especially as batch size, data-level parallelism, increases. Figure 4 depicts the execution time breakdown per operator with the majority of the time spent executing FC and SLS Caffe2 operators [75]. With a batch size of 8, SLS accounts for 37.2% and 50.6% of the total model execution time of RMC1-small and RMC1-large, respectively. Whereas for larger models represented by RMC2-small and RMC2-large, a more significant portion of the execution time goes into SLS (73.5%, 68.9%). Furthermore, the fraction of time spent on the embedding table operations increases with higher batch-size — 37.2% to 61.1% and 50.6% to 71.3% for RMC1-small and RMC1-large respectively. Note, the execution time of RMC2-large is $3.6\times$ higher than RMC1-large because RMC2 comprises a higher number of embedding tables. Embedding table sizes are expected to increase further for models used in industry [81].

D. Roofline Analysis

Applying the roofline model [70], we find *recommendation models lie in the memory bandwidth-constrained region, close*

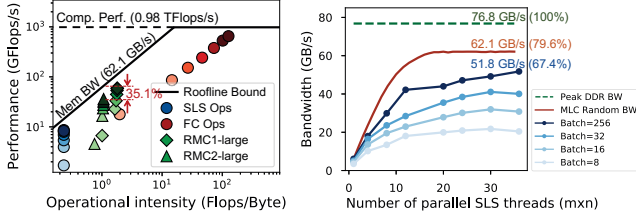


Fig. 5. Roofline of multi-threaded RMC1-large, RMC2-large sweeping batch size (1-256). Darker color indicates larger batch.

to the theoretical roofline performance bound. We construct a roofline describing the theoretical limits of the test system described in Section IV. We use Intel’s Memory Latency Checker (MLC)¹ to derive the memory bound. We derive the compute bound by sweeping the number of fused multiply-add (FMA) units in the processor and the operating frequency of the CPU (Turbo mode enabled).

Figure 5 presents the roofline data points for the models, RMC1 and RMC2, as well as their corresponding FC and SLS operators separately. We sweep batch size from 1 to 256 with darker colors indicating a larger batch size. We observe that the SLS operator has low compute but higher memory requirements; the FC portion of the model has higher compute needs; and the combined model is in between. SLS has low and fixed operational intensity across batch sizes, as it performs vector lookups and element-wise summation. FC’s operational intensity increases with batch size, as all requests in the batch share the same FC weights, increasing FC data reuse. With increasing batch size, the FC operator moves from the region under the memory-bound roofline to the compute-bound region. For the full model, we find RMC1 and RMC2 in the memory bound region, as the operational intensity is dominated by the high percentage of SLS operations. It also reveals that, with increasing batch size, the performance of SLS, as well as RMC1 and RMC2, is approaching the theoretical performance bound of the system.

More importantly, *our roofline analysis suggests that the performance of the recommendation model is within 35.1% of the theoretical performance bound and there is little room for further improvement without increasing system memory bandwidth.* By performing the embedding lookups and pooling operations before crossing the pin-limited memory interface, near-memory processing can exploit higher internal bandwidth of the memory system, thus effectively lifting up the roofline and fundamentally improving the memory bandwidth-constrained performance bound.

E. Memory Bandwidth of Production Configurations

Executing embedding operations on real systems can saturate memory bandwidth at high model-, operator- and data-level parallelism. Figure 6 depicts the memory bandwidth consumption as we increase the number of parallel SLS

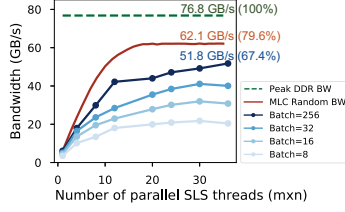


Fig. 6. Memory bandwidth saturation with increasing number of parallel SLS threads and batch sizes.

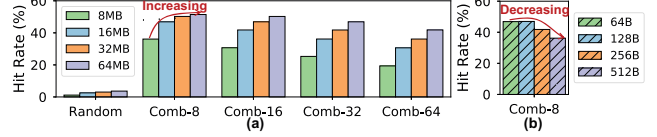


Fig. 7. (a) Temporal data locality sweeping cache capacity 8-64MB with fixed cacheline size of 64B; (b) Spatial data locality sweeping cacheline size 64-512B with fixed cache capacity 16MB.

threads for different batch sizes (blue curves). The green horizontal line represents the ideal peak bandwidth (76.8 GB/s, 4-channel, DDR4-2400) and the red curve is an empirical upper bound measured with Intel MLC [30]. We observe that memory bandwidth can be easily saturated by embedding operations especially as batch size and the number of threads increase. In this case, the memory bandwidth saturation point occurs (batch size = 256, number of SLS threads = 30) where more than 67.4% of the available bandwidth is taken up by SLS. In practice, a higher level of bandwidth saturation beyond this point becomes undesirable as memory latency starts to increase significantly [37]. *What is needed is a system that can perform the Gather-Reduce operation near memory such that only the final output from the pooling returns to the CPU.*

F. Embedding Table Locality Analysis

Prior work [75], [81] has assumed that embedding table lookups are random, however we show, *for traces from production traffic, there exists modest level of locality mostly due to temporal reuse.* While recommendation models are limited by memory performance generally, we wanted to study the memory locality to see if caching can improve performance. We evaluate both a random trace and embedding table (T1-T8) lookup traces from production workloads used by Eisenman et al. [10]. In production systems, one recommendation model contains tens of embedding tables and multiple models are co-located on a single machine. To mimic the cache behavior of a production system, we simulate the cache hit rate for multiple embedding tables co-located on one machine. In Figure 7(a), Comb-8 means that 8 embedding tables are running on the machine and the T1-T8 traces (each for a single embedding table) are interleaved for the 8 embedding tables. For Comb-16, Comb-32 and Comb-64 we multiply the 8 embedding tables 2, 4, and 8 times on the same machine, which also approximates larger models with 16, 32 and 64 embedding tables. We use the LRU cache replacement policy and 4-way set associative cache. We assume each embedding table is stored in a contiguous logical address space and randomly mapped to free physical pages.

To estimate the amount of temporal locality present, we sweep the cache capacity between 8-64MB with fixed cacheline size of 64B. In Figure 7(a), the random trace has a low hit rate of <5% representing the worst case locality. We see that the combined simulation of production traces is much higher than random with a hit rate between 20% and 60%. More importantly, hit rate increases as cache size increases. In Section III-D, we will show how optimizations to *RecNMP* can

¹Intel MLC [30] measures the bandwidth from the processor by creating threads that traverse a large memory region in random or sequential stride as fast as possible.

take advantage of this locality through table-aware packet scheduling and software locality hints from batch profiling.

Spatial locality can be estimated by sweeping the cacheline size of 64-512B with a fixed cache capacity of 16MB. Figure 7(b) illustrates this sweep for the Comb-8. We observe that as the cacheline size increases, in fact, hit rate decreases. In order to isolate the effect of increased conflict misses we run the same experiment on a fully-associative cache and observe similar trends of decreasing hit rate. Thus, we conclude that embedding table lookup operations have little spatial locality.

III. RECNP SYSTEM DESIGN

Considering the unique memory-bounded characteristics and the sparse and irregular access pattern of personalized recommendation, we propose *RecNMP*—a practical and lightweight near-memory processing solution to accelerate the dominated embedding operations. It is designed to maximize DRAM rank-level parallelism by computing directly and locally on data fetched from concurrently activated ranks.

First, we employ a minimalist style hardware architecture and embed specialized logic units and a rank-level cache to only support the SLS-family inference operators instead of general-purpose computation. The modified hardware is limited to the buffer chip within a DIMM without requiring any changes to commodity DRAM devices. Next, the sparse, irregular nature of embedding lookups exerts a high demand on command/address (C/A) bandwidth. This is addressed by sending a compressed instruction format over the standard memory interface, conforming to the standard DRAM physical pin-outs and timing constraints. Other proposed NMP solutions have employed special NMP instructions without addressing the C/A limitation of irregular and low spatial locality memory accesses pattern [22], [81]. We also present a hardware/software (HW/SW) interface for host-NMP coordination by adopting a heterogeneous computing programming model, similar to OpenCL [35]. Finally, we explore several HW/SW co-optimization techniques—*memory-side caching*, *table-aware scheduling* and *hot entry profiling*—that provide additional performance gains. These approaches leverage our observations from the workload characterization in the previous section.

A. Hardware Architecture

System overview. *RecNMP* resides in the buffer chip on the DIMM. The buffer chip bridges the memory channel interface from the host and the standard DRAM device interface, using data and C/A pins, as illustrated in Figure 8(a). Each buffer chip contains a *RecNMP* processing unit (PU) made up of a DIMM-NMP module and multiple rank-NMP modules. This approach is non-intrusive and scalable, as larger memory capacity can be provided by populating a single memory channel with multiple *RecNMP*-equipped DIMMs. Multiple DDR4 channels can also be utilized with software coordination.

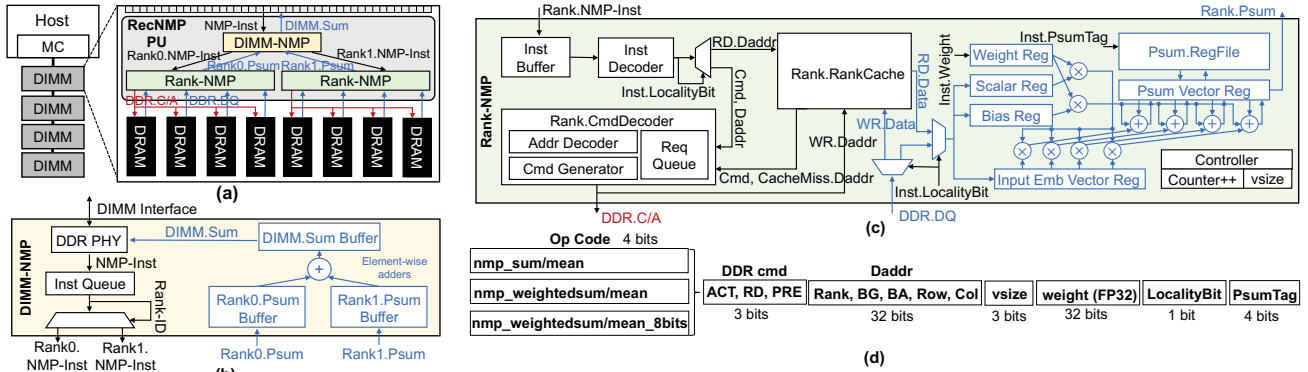
The host-side memory controller communicates with a *RecNMP* PU by sending customized compressed-format NMP instructions (NMP-Inst) through the conventional memory channel interface; the PU returns the accumulated embedding

pooling results (DIMM.Sum) to the host. Regular DDR4-compatible C/A and data signals (DDR.C/A and DDR.DQ) are decoded by the *RecNMP* PU from the NMP-Insts and then sent to all DRAM devices across all parallel ranks in a DIMM. By placing the logic at rank-level, *RecNMP* is able to issue concurrent requests to the parallel ranks and utilize, for SLS-family operators, the higher internal bandwidth present under one memory channel. Its effective bandwidth thus aggregates across all the parallel activated ranks. For example, in Figure 8(a), a memory configuration of 4 DIMMs $\times$ 2 ranks per DIMM could achieve $8\times$ higher internal bandwidth.

The DIMM-NMP module first receives a NMP-Inst through DIMM interface and then forwards it to the corresponding rank-NMP module based on the rank address. The rank-NMPs decode and execute the NMP-Inst to perform the local computation of the embedding vectors concurrently. We do not confine a SLS operation to a single rank but support aggregation across ranks within the PU. This simplifies the memory layout and increases bandwidth. DIMM-NMP performs the remaining element-wise accumulation of the partial sum vectors (PSum) from parallel ranks to arrive at the final result (DIMM.Sum). In the same fashion, Psums could be accumulated across multiple *RecNMP* PUs with software coordination. We will next dive into the design details on the DIMM-NMP and rank-NMP modules. While they are on the same buffer chip, having separate logical modules makes it easy to scale to DIMMs with a different number of ranks.

DIMM-NMP Module. To dispatch the NMP-Inst received from the DIMM interface, the DIMM-NMP module employs DDR PHY and protocol engine similar to the design of a conventional DIMM buffer chip relaying the DRAM C/A and DQ signals from and to the host-side memory controller. The instruction is multiplexed to the corresponding ranks based on the Rank-ID as shown in Figure 8(b). DIMM-NMP buffers the Psum vectors accumulated by each rank-NMP in its local registers and performs final summation using an adder tree before sending the final result back to the host via the standard DIMM interface. Depending on the memory system configuration, the number of ranks within a DIMM can vary, changing the number of inputs to the adder tree.

Rank-NMP Module. *RecNMP* uses the internal bandwidth on a DIMM to increase the effective bandwidth of embedding table operations, thus the majority of the logic is replicated for each rank. Three crucial functions are performed by the rank-NMP module—translating the NMP-Inst into low-level DDR C/A commands, managing *memory-side caching* and computing SLS-family operators locally. As illustrated in Figure 8(c), the NMP-Inst is decoded to control signals and register inputs. To address C/A bus limitations, all of the DDR commands for a single SLS vector is embedded in one NMP-Inst. Three fields in NMP-Inst (Figure 8(d))—DDR cmd (the presence/absence of {ACT, RD, PRE} with bit 1/0), vector size (vsize), and DRAM address (Daddr)—determine the DDR command sequence and the burst length. These are fed to the local command decoder (Rank.CmdDecoder) to generate standard DDR-style ACT/RD/PRE commands to communicate



with DRAM devices. The tags are set at runtime by the host-side memory controller based on the relative physical address location of consecutive embedding accesses. This keeps the CmdDecoder in rank-NMP lightweight, as the host-side memory controller has performed the heavy-lifting tasks of request reordering, arbitration, and clock and refresh signal generation. If a 128B vector (vsize=2) requires ACT/PRE from a row buffer miss, the command sequence to DRAM devices for the NMP-Inst is {PRE, ACT Row, RD Col, RD Col+8} decoded from {ACT, RD, PRE} and vsize tags.

Our locality analysis in Section II shows that the modest temporal locality within some embedding tables as vectors are reused. The operands of each SLS-family operator vary so caching the final result in the DIMM or CPU will be ineffective. We incorporate a memory-side cache (RankCache) in each rank-NMP module to exploit the embedding vectors reuse. The RankCache in *RecNMP* takes hints from the LocalityBit in the NMP-Inst to determine whether an embedding vector should be cached or bypassed. The detailed method to generate the LocalityBit hint through hot entry profiling will be explained in Section III-D. Entries in RankCache are tagged by the DRAM address field (Daddr). If the LocalityBit in the NMP-Inst indicates low locality, the memory request bypasses the RankCache and is forwarded to Rank.CmdDecoder to initiate a DRAM read. Embedding tables are read-only during inference, so this optimization does not impact correctness.

The datapath in the rank-NMP module supports a range of SLS-family operators. The embedding vectors returned by the RankCache or DRAM devices are loaded to the input embedding vector registers. For weighted sum computation, the weight registers are populated by the weight fields from the NMP-Inst. For quantized operators such as the SLS-8bits operator, the dequantized parameters *Scalar* and *Bias* are stored with the embedding vectors and can be fetched from memory to load to the Scalar and Bias registers. The Weight and Scalar/Bias registers are set to be 1 and 1/0 during execution of non-weighted and non-quantized SLS operators. The PsumTag decoded from the NMP-Inst is used to identify the embedding vectors belonging to the same pooling operations, as multiple poolings in one batch for one embedding table could be served in parallel. The controller counter, vector

size register, and final sum registers in the both the DIMM-NMP and rank-NMP modules are all memory-mapped, easily accessible and configurable by the host CPU.

B. C/A Bandwidth Expansion

Although the theoretical aggregated internal bandwidth of *RecNMP* scales linearly with the number of ranks per channel, in practice, the number of concurrently activated ranks is limited by the C/A bandwidth. Due to frequent row buffer misses/conflicts from low spatial locality, accessing the embedding table entries in memory requires a large number of ACT and PRE commands. The reason is that the probability of accessing two embedding vectors in the same row is quite low, as spatial locality only exists in continuous DRAM data burst of one embedding vector. In production, embedding vector size ranges from 64B to 256B with low spatial locality, resulting in consecutive row buffer hits in the narrow range of 0 to 3.

To fully understand the C/A bandwidth limitation, we analyze the worst-case scenario when the embedding vector size is 64B. A typical timing diagram is presented in Figure 9(a). It shows an ideal sequence of bank-interleaved DRAM reads that could achieve one consecutive data burst. In this burst mode, the *ACT* command first sets the row address. Then the *RD* command is sent accompanied by the column address. After t_{RL} DRAM cycles, the first set of two 64-bit data (DQ0 and DQ1) appear on the data bus. The burst mode lasts for 4 DRAM cycles (burst length = 8) and transmits a total of 64B on the DQ pins at both rising and falling edges of the clock signal. Modern memory systems employ bank interleaving, therefore in the next burst cycle (4 DRAM cycles), data from a different bank can be accessed in a sequential manner. In this ideal bank interleaving case, every 64B data transfer takes 4 DRAM cycles and requires 3 DDR commands (*ACT*/*RD*/*PRE*) to be sent over the DIMM C/A interface, this consumes 75% of the C/A bandwidth. Activating more than one bank concurrently would require issuing more DDR commands, thus completely exhausting the available C/A bandwidth of conventional memory interface.

To overcome C/A bandwidth limitation, we propose a customized NMP-Inst with a compressed format of DDR commands to be transmitted from memory controller to Rec-NMP PUs. Figure 9(b) illustrates the timing diagram of

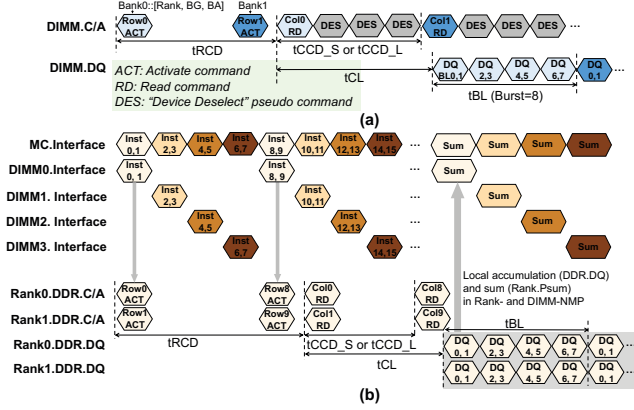


Fig. 9. Timing diagram of (a) ideal DRAM bank interleaving read operations; (b) The proposed *RecNMP* concurrent rank activation.

interleaving NMP-Inst to a 4 DIMMs $\times$ 2 Ranks per DIMM memory configuration. Eight NMP-Insts can be transferred between memory controller and DIMMs interfaces in 4 DRAM data burst cycles on double data rate. In low spatial locality case (64B embedding vector and one NMP-Inst per vector) and ideal bank interleaving, we could potentially activate 8 parallel ranks to perform $8 \times 64B$ lookups concurrently in 4 DRAM data burst cycles. Although customized instructions have been proposed before [5], [22], [81], our solution is the first one to directly deal with the C/A bandwidth limitation using DDR command compression that enables up to $8 \times$ bandwidth expansion for small-sized embedding vectors (i.e. 64B) with low spatial locality. Higher expansion ratio can be achieved with larger vector size.

C. Programming Model and Execution Flow

Like previous NMP designs [22], [32], *RecNMP* adopts a heterogeneous computing programming model (e.g. OpenCL), where the application is divided into host calls running on the CPU and NMP kernels being offloaded to *RecNMP* PUs. NMP kernels are compiled into packets of NMP-Insts and transmitted to each memory channel over the DIMM interface to *RecNMP* PUs. Results of NMP kernels are then transmitted back to the host CPU. In Figure 8(d), each 79-bit NMP-Inst contains distinctive fields that are associated with different parameters in an embedding operation, locality hint bit (LocalityBit) and pooling tags (PsumTag) passed between the HW/SW interface. The proposed NMP-Inst format can fit within the standard 84-pin C/A and DQ interface.

Using a simple SLS function call in Figure 10(a) as an example, we walk through the execution flow of the proposed *RecNMP* programming model. First, memory is allocated for SLS input and output data, and is marked up as either Host (cacheable) or NMP (non-cacheable) regions to simplify memory coherence between the host and *RecNMP*. Variables containing host visible data, such as the two arrays *Indices* and *Lengths*, are initialized and loaded by the host and are cacheable in the host CPU's cache hierarchy. The embedding table (Emb) in memory is initialized by the host as a host non-cacheable NMP region using a non-temporal hint (NTA) [31].

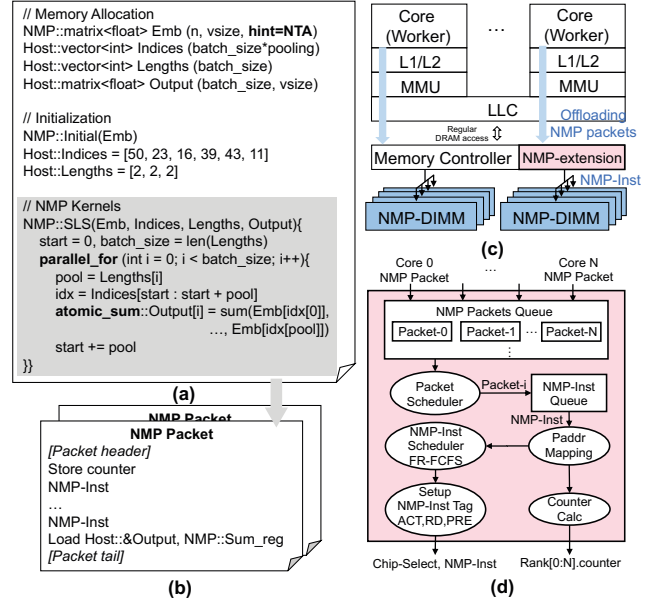


Fig. 10. (a) *RecNMP* SLS example code; (b) NMP packet; (c) NMP kernel offloading; (d) NMP-enabled memory controller.

Next, the code segment marked as a NMP kernel is compiled to packets of NMP-Insts (Figure 10(b)). A single SLS NMP kernel containing one batch of embedding poolings can be split into multiple NMP packets, with each packet having one or more pooling operations. The NMP-Insts belonging to different embedding poolings in one NMP packet are tagged by PsumTag, and the maximum number of poolings in one packet is determined by the number of bits of the PsumTag. We use a 4-bit PsumTag in our design. At runtime, the NMP kernel is launched by the host with special hardware/driver support to handle NMP packet offloading; access to the memory management unit (MMU) to request memory for NMP operations; and the virtual memory system for logical-to-physical addresses translation (Figure 10(c)). The offloaded NMP packets bypass L1/L2 and eventually arrive at the host-side memory controller with an NMP extension. To avoid scheduling the NMP packets out-of-order based on FR-FCFS policy, the NMP extension of the memory controller includes extra scheduling and arbitration logic.

As illustrated in Figure 10(d), the memory controller with the NMP extension receives concurrent NMP packets from parallel execution of multiple host cores, which are stored in a queue. Once scheduled, each NMP packet is decoded into queued NMP-Insts. Physical-to-DRAM address mapping is then performed and a FR-FCFS scheduler reorders the NMP-Insts within a packet only and not between packets. Instead of sending direct DDR commands, ACT/RD/PRE actions are compressed into the 3-bit DDR_cmd field in the NMP-Inst. The host-side memory controller also calculates the correct accumulation counter value to configure the memory-mapped control registers in the *RecNMP* PU. Finally, after the completion of all the counter-controlled local computation inside the *RecNMP* PU for one NMP packet, the final summed

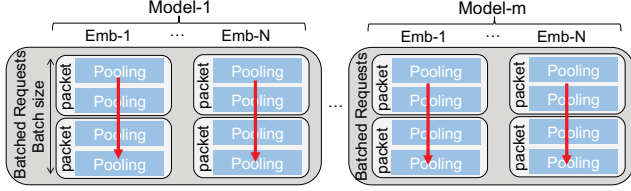


Fig. 11. NMP packet scheduling scheme that prioritizes batch of single table.

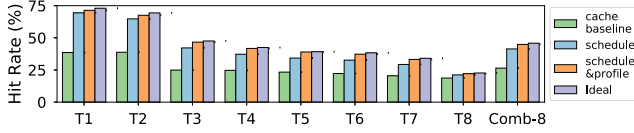


Fig. 12. Hit rate of 1MB cache without optimization, with table-aware packet scheduling optimization, with both table-aware packet scheduling and hot entry profiling optimization, and ideal case without interference.

result is transmitted over the DIMM interface and returned to the *Output* cacheable memory region visible to the CPU.

D. HW/SW Co-optimization

Our locality analysis of production recommendation traffic in Section II-F illustrates intrinsic temporal reuse opportunities in embedding table lookups. We propose memory-side caches (RankCache) inside rank-NMP modules. To extract more performance from memory-side caching, we explore two additional HW/SW co-optimization techniques. This locality-aware optimization results in 33.7% memory latency improvement and 45.8% memory access energy saving (detailed performance benefits will be presented in Section V).

First, to preserve the intrinsic locality from embedding lookups residing in one table, we propose to prioritize scheduling NMP packets from a single batch requests to the same embedding table together – *table-aware packet scheduling*. In production workloads, the memory controller receives NMP packets from parallel SLS threads with equal scheduling priority. The intra-embedding table temporal locality is not easily retained because of the interference from lookup operations of multiple embedding tables. This locality can be further degraded when multiple recommendation models are co-located. Therefore, as illustrated in Figure 11, we propose an optimized table-aware NMP packet scheduling strategy to exploit the intrinsic temporal locality within a batch of requests by ordering packets from the same embedding table in one batch first, allowing the embedding vectors to be fetched together, thereby retaining the temporal locality. SLS operators access separate embedding tables as running in parallel threads, the mechanics of our implementation comes from the thread-level memory scheduler [61].

Next, we propose another optimization technique – *hot entry profiling*, built on top of the observation that a small subset of embedding entries exhibit relatively higher reuse characteristics. We profile the vector of indices used for embedding table lookup in an NMP kernel and mark the entries with high locality by explicitly annotating NMP-Insts with a *LocalityBit*. NMP-Inst with *LocalityBit* set will be cached in the RankCache; otherwise, the request will bypass

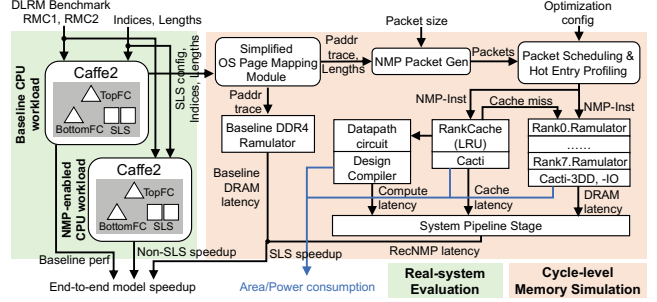


Fig. 13. *RecNMP* experimental methodology.

the RankCache. This hot entry profiling step can be performed before model inference and issuing SLS requests and only costs $< 2\%$ of total end-to-end execution time. We profile the indices of each incoming batch of embedding lookups and set *LocalityBit* if the vectors are accessed $> t$ times within the batch. Infrequent ($< t$ times) vectors will bypass the RankCache and are read directly from the DRAM devices. We sweep the threshold t and pick the value with the highest cache hit rate to use in our simulation. This hot entry profiling optimization reduces cache contention and evictions caused by the less-frequent entries in the RankCache.

Figure 12 depicts the hit rate improvement when the different optimizations are applied. Comb-8 indicates the overall hit rate at model level of 8 embedding tables (T1-T8). To gain more insights, we investigate the hit rate of embedding tables (T1 to T8) in Comb-8. The ideal bar indicates the theoretical hit rate with an infinitely sized cache. With the proposed co-optimization, the measured hit rate closely approaches the ideal case across the individual embedding tables, even for the trace with limited locality (T8), illustrating the proposed technique can effectively retain embedding vectors with high likelihood of reuse in RankCache.

IV. EXPERIMENTAL METHODOLOGY

Our experimental setup combines real-system evaluations with cycle-level memory simulations, as presented in Figure 13. For real-system evaluations, we run production-scale recommendation models on server-class CPUs found in the data center. This allows us to measure the impact of accelerating embedding operations as well as the side-effect of improved memory performance of FC operations on end-to-end models. Cycle-level memory simulations allow us to evaluate the design tradeoffs when DRAM systems are augmented with *RecNMP*. Table I summarizes the parameters and configurations used in the experiments. We run experiments on an 18-core Intel Skylake with DDR4 memory. The DRAM simulation used standard DDR4 timing from a Micron datasheet [51].

Real-system evaluation. We configure the DRLM benchmark with the same model parameters and traces in Figure 2(b) and Section II. The workload characterization (Section II) and real-system experiments (Section V) are performed on single socket Intel Skylake servers, specifications in Table I.

Cycle-level memory simulation. We build the *RecNMP* cycle-level simulation framework with four main compo-

TABLE I
SYSTEM PARAMETERS AND CONFIGURATIONS

Real-system Configurations			
Processor	18 cores, 1.6 GHz	L1I/D	32 KB
L2 cache	1 MB	LLC	24.75 MB
DRAM	DDR4-2400MHz 8Gb × 8, 64 GB, 4 Channels × 1 DIMM × 2 Ranks, FR-FCFS 32-entry RD/WR queue, Open policy, Intel Skylake address mapping [66]		
DRAM Timing Parameters			
tRC=55, tRCD=16, tCL=16, tRP=16, tBL=4 tCCD_S=4, tCCD_L=6, tRRD_S=4, tRRD_L=6, tFAW=26			
Latency/Energy Parameters			
DDR Activate = 2.1nJ, DDR RD/WR = 14pJ/b, Off-chip IO = 22pJ/b RankCache RD/WR = 1 cycle, 50pJ/access, FP32 adder = 3 cycles, 7.89pJ/Op, FP32 mult = 4 cycles, 25.2pJ/Op			

nents: (1) physical addresses mapping module; (2) packet generator; (3) locality-aware optimizer; and (4) a cycle-accurate model of a RecNMP PU consisting of DRAM devices, RankCache, arithmetic and control logic. We use Ramulator [80] to conduct cycle-level evaluations of DDR4 devices. On top of Ramulator, we build a cycle-accurate LRU cache simulator for RankCache and model of the 4-stage pipeline in the rank-NMP module. Cacti [58] is used to estimate the access latency and area/energy of RankCache. The hardware implementation used to estimate the latency, area and power of the arithmetic logic is built from Synopsys Design Compiler with a commercial 40nm technology library. To estimate the DIMM energy, we use Cacti-3DD [41] for DRAM devices and Cacti-IO [59] for off-chip I/O at the DIMM level.

During simulation we emulate the scheduling packet generation steps taken by the software stack and the memory controller. First, we apply a standard page mapping method [50] to generate the physical addresses from a trace of embedding lookups by assuming the OS randomly selects free physical pages for each logical page frame. This physical address trace is fed to Ramulator to estimate baseline memory latency. For *RecNMP* workloads, the packet generator divides the physical address trace into packets of NMP-Insts that are sent to the cycle-accurate model. Next, when evaluating systems with HW/SW co-optimizations, the locality-aware optimizer performs table-aware packet scheduling and hot entry profiling and decides the sequence of NMP-Insts. *RecNMP* activate all memory ranks in parallel and traditional DRAM bank-interleaving is also used. For each NMP packet, performance is determined by the slowest rank that receives the heaviest memory request load. Rank-NMP and DIMM-NMP logic units are pipelined to hide the latency of memory read operations. The total latency of *RecNMP* includes extra DRAM cycles during initialization to configure the accumulation counter and the vector size register and a cycle in the final stage to transfer the sum to the host. The latency, in DRAM cycles, of the major components including RankCache, rank-NMP logic performing weighted-partial sum and final sum are in Table I.

V. EVALUATION RESULTS

This section presents a quantitative evaluation of *RecNMP* and shows it accelerates end-to-end personalized rec-

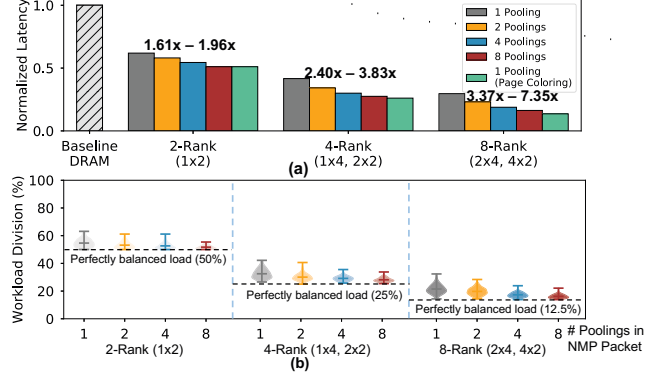


Fig. 14. (a) Normalized latency of RecNMP-base to the baseline DRAM with different memory configuration (DIMM × Rank) and NMP packet size; (b) Distribution of rank-level load imbalance for 2-, 4-, and 8-rank systems.

ommendation inference by up to 4.2×. We first present the latency improvement of the offloaded SLS operators on a baseline system before analyzing different optimizations including placement with page coloring, memory-side caching, table-aware packet scheduling and hot-entry profiling. We compare *RecNMP* with state-of-the-art NMP systems TensorDIMM and Chameleon [22], [81]. We also analyze the effect of *RecNMP* on co-located FC operators. Finally, an end-to-end evaluation of throughput improvement and energy savings at the model level and the area/power overhead is presented.

A. SLS Operator Speedup

In theory, because *RecNMP* exploits rank-level parallelism, speedup will scale linearly with the number of ranks and number of DIMMs in a system. Therefore, we choose four memory channel configurations (# of DIMMs × # of ranks per DIMM) that correspond to 1 × 2, 1 × 4 and 2 × 2, and 4 × 2 to demonstrate a range of system implementations.

Basic *RecNMP* design without RankCache. We start by evaluating *RecNMP* without a RankCache (RecNMP-base). In addition to varying the DIMM/rank configuration, we sweep the number of poolings in one NMP packet, where one pooling, in DLRM, is the sum of 80 embedding vectors. In Figure 14(a), we find 1) SLS latency indeed scales linearly as we increase the number of active ranks in a channel; 2) latency also decreases when there are more pooling operations in an NMP packet. The variation we observe, as well as the performance gap observed between the actual speedup and the theoretical speedup (2× for 2-rank, 4× for 4-rank, and 8× for 8-rank systems) is caused by the uneven distribution of embedding lookups across the ranks. As the ranks operate in parallel, the latency of the SLS operation is determined by the slowest rank, the rank that runs more embedding lookups. Figure 14(b) shows the statistical distribution of fraction of the work run on the slowest rank. When the NMP packet has fewer NMP-Insts, the workload distributes more unevenly, resulting in a longer tail that degrades average speedup.

To address the load imbalance, we experiment with software methods to allocate an entire embedding table to the same rank. One software approach to perform such data layout op-

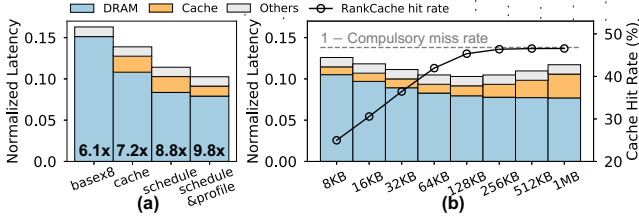


Fig. 15. (a) Normalized latency of RecNMP-cache and RecNMP-opt with schedule and hot-entry profile optimization to the baseline DRAM system; (b) Cache size sweep effects in RecNMP-opt.

timization is page coloring [78]. As indicated in Figure 14(a), page coloring could achieve $1.96\times$, $3.83\times$ and $7.35\times$ speedup in 2-rank, 4-rank and 8-rank system compared with the DRAM baseline. The specific page coloring mechanism can be implemented in the operating system by assigning a fixed color to the page frames used by an individual embedding table. The virtual memory system would need to be aware of the DRAM configuration to allocate pages of the same color to physical addresses that map to the same rank. This data layout optimization can lead to near-ideal speedup, but it requires maintaining high model- and task-level parallelism such that multiple NMP packets from different SLS operators can be issued simultaneously to all the available ranks.

RecNMP with RankCache and co-optimization. Memory-side caching at the rank-level with table-aware packet scheduling and hot entry profiling is one of the notable features of *RecNMP*; these optimizations are described in Section III-D. Figure 15(a) depicts the performance benefits (i.e. latency reduction) enabled by applying different optimization techniques: 1) adding a RankCache, 2) scheduling accesses to the same table together, 3) adding a cachability hint bit from software. Using a configuration with 8-ranks 8 poolings per packet, we observe 14.2% latency improvement by adding a 128KB RankCache and an additional 15.4% improvement by prioritizing the scheduling of NMP packets from the same table and batch. In the final combined optimization, *schedule + profile*, we pass cacheability hint after profiling the indices in the batch which reduces cache contention and allows low-locality requests not marked for caching to bypass the RankCache, delivering another 7.4% improvement. The total memory latency speedup achieved by offloading SLS to an optimized design (RecNMP-opt) is $9.8\times$.

In Figure 15(b), we sweep RankCache capacity from 8KB to 1MB and display how cache size affects the normalized latency and cache hit rate. When RankCache is small (e.g. 8KB), the low cache hit rate (e.g. 24.9%) leads to high DRAM access latency. The performance reaches the optimal design point at 128KB. Further increase of cache size has marginal improvement on hit rate, since it already reaches the compulsory limit in the trace. Yet it incurs longer cache access latency and degrades overall performance.

Performance comparison. We compare *RecNMP* with state-of-the-art NMP designs such as Chameleon [22] and TensorDIMM [81]. Both are DIMM-based near-memory processing solutions. TensorDIMM scales the embedding operation

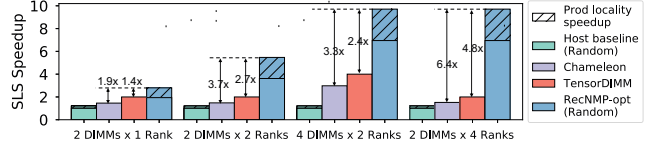


Fig. 16. Comparison between Host baseline, RecNMP-opt, TensorDIMM [81] and Chameleon [22] with both random and production traces

performance linearly with the number of parallel DIMMs. Since non-SLS operators are accelerated by GPUs in TensorDIMM, which is orthogonal to near-memory acceleration techniques, we only compare its memory latency speedup with *RecNMP*. Chameleon does not directly support embedding operations. We estimate its performance of Chameleon by simulating the temporal and spatial multiplexed C/A and DQ timing of Chameleon’s NDA accelerators. In Figure 16, as *RecNMP* exploits rank-level parallelism, its performance scales when either the number of DIMMs and ranks increase, whereas Chameleon and TensorDIMM only scale by increasing the number of DIMMs. This is evident as we sweep the memory channel configuration. When we increase the number of ranks per-DIMM, *RecNMP* can deliver $3.3\text{--}6.4\times$ and $2.4\text{--}4.8\times$ better performance than Chameleon and TensorDIMM.

It is also worth noting that *RecNMP* has performance advantages ($1.9\times$ and $1.4\times$) even in configurations with one rank per DIMM, thanks to the memory-side caching, table-aware packet scheduling, and hot-entry profiling optimization techniques. Neither Chameleon nor TensorDIMM includes a memory-side cache to explicitly take advantage of the available locality in the memory access patterns, hence their performance, with respect to memory latency, is agnostic to traces with different amounts of data reuse. In contrast, *RecNMP* design can extract 40% more performance (shown as shaded) from production traces when compared to fully random traces.

B. FC Operator Speedup

Although *RecNMP* is designed to accelerate the execution of SLS operators, it can also improve FC performance by alleviating cache contention caused by model co-location. As the degree of data-level parallelism increases, the FC weights brought into the cache hierarchy have higher reuse, normally resulting in fewer cache misses. However, when co-located with other models, reusable FC data are often evicted early from the cache by SLS data, causing performance degradation.

Figure 17 shows the degree of performance degradation on the co-located FC operations. The amount of performance degradation experienced by the FC layers varies by the FC sizes, the degree of co-location, and the pooling values. When examining the FC performance in baseline systems, we observe worsening FC performance with larger FC weights at higher co-location degrees and higher pooling values. *RecNMP* effectively reduces the pressure from the cache contention, we show the base *RecNMP* design but RecNMP-opt impacts FC performance equally as it offloads the same SLS computation. This beneficial effect ranging from 12% to 30% is more pronounced for larger FCs whose weight parameters exceed the capacity of the L2 cache and reside mainly inside

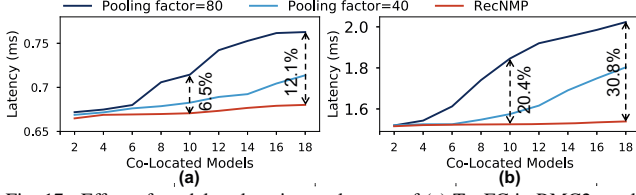


Fig. 17. Effect of model co-location on latency of (a) TopFC in RMC2-small model; (b) TopFC in RMC2-large model.

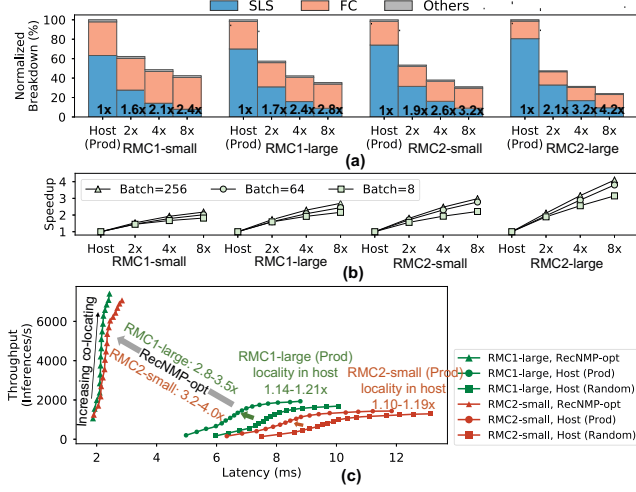


Fig. 18. (a) Single end-to-end speedup of recommendation inference with 2-rank, 4-rank and 8-rank *RecNMP* systems; (b) Single model speedup with different batch size; (c) Host and *RecNMP*-opt co-located model latency-throughput tradeoff.

the LLC cache. For smaller FCs whose working set fits inside the L2 cache (e.g. all BottomFC and RMC1’s TopFC), the relative improvement is comparatively lower ($\sim 4\%$).

C. End-to-end Model Speedup

Throughput improvement. To estimate the improvement of end-to-end recommendation inference latency, we calculate the total speedup by weighting the speedup of both SLS and non-SLS operators. We measure model-level speedup across all four representative model configurations, shown in Figure 18(a). Not surprisingly, the model that spends the most time running SLS operators (RMC2-large) receives the highest speedup. In Figure 18(b), the performance improvement obtained by *RecNMP* varies with batch size. In general, the model-level speedup increases with a larger batch size, as the proportion of time spent in accelerated SLS operators grows.

Figure 18(c) looks at the overall effect of increasing co-location in the presence of random or production traces for both the CPU baseline and our proposed *RecNMP* solution. Co-location generally increases the system throughput at the cost of degrading latency. Compared to random traces, the locality present in production traces improves performance. However, this locality performance “bonus” wears off as the level of model co-location increases due to the cache interference from the growing number of embedding tables in multiple models. Applying *RecNMP* in a 8-rank system results in 2.8-3.5 $\times$ and 3.2-4.0 $\times$ end-to-end speedup of RMC1-large and RMC2-small as the number of co-located models increases,

TABLE II
SUMMARY OF *RecNMP* DESIGN OVERHEAD

	RecNMP PU		Chameleon [22] (8 CGRA accelerators)
	RecNMP-base w/o RankCache	RecNMP-opt with RankCache	
Area (mm ²)	0.34	0.54	8.34
Power (mW)	151.3	184.2	3138.6-3251.8

because the fraction of SLS latency rises. The improvement of both latency and throughput enabled by *RecNMP* is clearly observed compared to the baseline system.

Memory energy savings. Comparing with the baseline DRAM system, *RecNMP* provide 45.8% memory energy saving. *RecNMP* saves the energy from the reduced data movement between the processor and the memory by performing local accumulation near DRAM devices and the leakage saving from reduced latency. In addition, by incorporating memory-side caching and applying co-optimization techniques to improve RankCache hit rate, *RecNMP* achieves extra energy savings by reducing the number of DRAM accesses.

Area/power overhead. We estimate *RecNMP* design overhead assuming 250MHz clock frequency and 40nm CMOS technology. The area and power numbers are derived from Synopsys Design Compiler (DC) for the arithmetic and control logic and Cacti [58] for SRAM memory (i.e. RankCache). Table II summarizes the overhead of each *RecNMP* processing unit for both the basic configuration without cache and the optimized configuration with cache optimization.

Compared with Chameleon, which embeds 8 CGRA cores per DIMM, our *RecNMP* PU consumes a fraction of the area (4.1%, 6.5% for *RecNMP*-base and *RecNMP*-opt) and power (4.6-5.9%). When scaling *RecNMP* PUs to multiple ranks in the DIMM, the total area and power will grow linearly, but it also translates to linearly-scaled embedding speedup. Given that a single DIMM consumes 13W [81] and a typical buffer chip takes up 100mm² [62], *RecNMP* incurs small area/power overhead that can easily be accommodated without requiring any changes to the DRAM devices.

VI. RELATED WORK

Performance characterization of recommendation models. Recent publications have discussed the importance and scale of personalized recommendation models in data center [1], [14], [49], [64], [65], [75], [77]. Compared to CNNs, RNNs, and FCs [12], [69], [72], [82], the analysis demonstrates how recommendation models have unique storage, memory bandwidth, and compute requirements. For instance, [75] illustrates how Facebook’s personalized recommendation models are dominated by embedding table operations. To the best of our knowledge, *RecNMP* is the first to perform locality study using production-scale models with representative embedding traces.

DRAM-based near-memory and near-data acceleration. Many prior works explore near-memory processing using 3D/2.5D-stacked DRAM technology (e.g. HMC/HBM) [5], [13], [17], [39], [42], [43], [45], [46], [48], [56], [67]. Due to their limited memory capacity (16-32GB) and high

cost of ownership, these schemes are not suitable for large-scale deployment of recommendation models (10s to 100s of GBs) in production environment. Chameleon [22] introduces a practical approach to performing near-memory processing by integrating CGRA-type accelerators inside the data buffer devices in a commodity LRDIMM [22]. Unlike Chameleon’s DIMM-level acceleration, *RecNMP* exploits rank-level parallelism with higher speedup potential. *RecNMP* also employs a lightweight NMP design tailored to sparse embedding operators with much lower area and power overheads than CGRAs.

Supporting error correction (e.g. ECC) is important for reliable operation. Since *RecNMP* keeps the rank as the minimal logical device, all DRAM chips in a rank being addressed simultaneously, this preserves design simplicity for ECC operations (8 data chips and 1 parity chip). Alternatively, error detection/correction logic could be placed on the buffer chip before the compute logic, incurring very low hardware overhead, such as [9], [16], [26]–[28]. General ECC designs tailored for near memory processing remain an open question.

System optimization for memory-constrained learning models. Sparse embedding representations have been commonly employed to augment deep neural network (DNN) models with external memory to memorize previous history. Eisenman et al. explore the use of NVMs for large embedding storage [10]. Although the proposed techniques result in $2 - 3\times$ improvement of effective NVM read bandwidth ($2.3GB/s$), it remains far below typical DRAM bandwidth ($76.8GB/s$) and cannot fundamentally address the memory bandwidth bottleneck in recommendation models. MnnFast targets optimization for memory-augmented neural network and proposes a dedicated embedding cache to eliminate the cache contention between embedding and inference operations [23]. However, these techniques do not directly apply to personalized recommendation consisting order-of-magnitude larger embedding tables. TensorDIMM [81] proposes a custom DIMM module enhanced with near-memory processing cores for embedding and tensor operations in deep learning. The address mapping scheme in TensorDIMM interleaves consecutive 64B within each embedding vector across the DIMM modules. Its performance thus scales at the DIMM level and relies on the inherent high spatial locality of large embedding vectors, it is unable to apply to this approach to small vectors (e.g. 64B). Given the same memory configuration, our design can outperform TensorDIMM in memory latency speedup by extracting additional performance gains from rank-level parallelism and memory-side caching optimizations. The introduction of a customized compressed NMP instruction in *RecNMP* also fundamentally addresses the C/A bandwidth constraints, without the restrictions on small embedding vectors as imposed by TensorDIMM.

VII. CONCLUSION

We propose *RecNMP*—a practical and scalable near-memory solution for personalized recommendation. We perform a systematic characterization of production-relevant recommendation models and reveal its performance bottleneck.

A light-weight, commodity DRAM compliant design, *RecNMP* maximally exploits rank-level parallelism and temporal locality of production embedding traces to achieve up to $9.8\times$ performance improvement of sparse embedding operation (carried out by the SLS-family operators). Offloading SLS also offers alleviated cache contention for the non-SLS operators that remain in the CPU, resulting in up to 30% latency reduction for co-located FC operators. Overall, our system-level evaluation demonstrates that *RecNMP* offers up to $4.2\times$ throughput improvement and 45.8% memory energy saving with representative production-relevant model configurations.

REFERENCES

- [1] “Breakthroughs in Matching and Recommendation Algorithms by Alibaba.” [Online]. Available: https://www.alibabacloud.com/blog/breakthroughs-in-matching-and-recommendation-algorithms-by-alibaba_593976
- [2] “Caffe2.” [Online]. Available: <https://caffe2.ai///>
- [3] Ali Shafiee, Anirban Nag, Naveen Muralimanohar, Rajeev Balasubramanian, John Paul Strachan, Miao Hu, R. Stanley Williams, Vivek Srikumar, “ISAAC: A Convolutional Neural Network Accelerator with In-Situ Analog Arithmetic in Crossbars,” in *ISCA*, 2016, pp. 14–26.
- [4] Amazon Personalize, <https://aws.amazon.com/personalize/>.
- [5] Amin Farmahini-Farahani, Jung Ho Ahn, Katherine Morrow, and Nam Sung Kim, “NDA: Near-DRAM Acceleration Architecture Leveraging Commodity DRAM Devices and Standard Memory Modules,” in *HPCA*, 2015.
- [6] Amir Yazdanbakhsh, Kambiz Samadi, Hadi Esmaeilzadeh, Nam Sung Kim, “GANAX: A Unified SIMD-MIMD Acceleration for Generative Adversarial Network,” in *ISCA*, 2018.
- [7] Angshuman Parashar, Minsoo Rhu, Anurag Mukkara, Antonio Puglielli, Rangharajan Venkatesan, Bruce Khailany, Joel Emer, Stephen W. Keckler, William J. Dally, “SCNN: An accelerator for compressed-sparse convolutional neural networks,” in *ISCA*, 2017.
- [8] Animesh Jain, Amar Phanishayee, Jason Mars, Lingjia Tang, Gennady Pekhimenko, “Gist: Efficient Data Encoding for Deep Neural Network Training,” in *ISCA*, 2018.
- [9] Aniruddha N. Udiipi, Naveen Muralimanohar, Rajeev Balasubramanian, Al Davis, Norman P. Jouppi, “LOT-ECC: Localized and Tiered Reliability Mechanisms for Commodity Memory Systems,” in *ISCA*, 2012.
- [10] Assaf Eisenman, Maxim Naumov, Darryl Gardner, Misha Smelyanskiy, Sergey Pupyrev, Kim Hazelwood, Asaf Cidon, Sachin Katti, “Bandana: Using non-volatile memory for storing deep learning models,” in *SysML*, 2019.
- [11] Ben Feinberg, Shibo Wang and Engin Ipek, “Making Memristive Neural Network Accelerators Reliable,” in *HPCA*, 2018.
- [12] Brandon Reagen, Paul Whatmough, Robert Adolf, Saketh Rama, Hyunkwang Lee, Sae Kyu Lee, José Miguel Hernández-Lobato, Gu-Yeon Wei, David Brooks, “Minerva: Enabling low-power, highly-accurate deep neural network accelerators,” in *ISCA*. IEEE, 2016, pp. 267–278.
- [13] Byungchul Hong, Gwangsun Kim, Jung Ho Ahn, Yongkee Kwon, Hongsik Kim, John Kim, “Accelerating linked-list traversal through near-data processing,” in *PACT*, 2016, pp. 113–124.
- [14] Carole-Jean Wu, Robin Burke, Ed H. Chi, Joseph Konstan, Julian McAuley, Yves Raimond, Hao Zhang, “Developing a Recommendation Benchmark for MLPerf Training and Inference,” in *arXiv preprint arXiv:2003.07336*, 2020.
- [15] Christopher De Sa, Matthew Feldman, Christopher Ré, Kunle Olukotun, “Understanding and Optimizing Asynchronous Low-Precision Stochastic Gradient Descent,” in *ISCA*, 2017.
- [16] Doe Hyun Yoon, Mattan Erez, “Virtualized and Flexible ECC for Main Memory,” in *ASPLOS*, 2010.
- [17] Duckhwan Kim, Jaeha Kung, Sek Chai, Sudhakar Yalamanchili, Saibal Mukhopadhyay, “Neurocube: A Programmable Digital Neuromorphic Architecture with High-Density 3D Memory,” in *ISCA*, 2016.
- [18] Eunhyeok Park, Dongyoung Kim, Sungjoo Yoo, “Energy-Efficient Neural Network Accelerator Based on Outlier-Aware Low-Precision Computation,” in *ISCA*, 2018.

- [19] Fortune, <https://fortune.com/2019/04/30/artificial-intelligence-walmart-stores/>.
- [20] Google Cloud Platform, <https://cloud.google.com/solutions/recommendations-using-machine-learning-on-compute-engine>.
- [21] Guorui Zhou, Xiaoqiang Zhu, Chengru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, Kun Gai, "Deep Interest Network for Click-Through Rate Prediction," in *KDD*, 2018, pp. 1059–1068.
- [22] Hadi Asghari-Moghaddam, Young Hoon Son, Jung Ho Ahn, Nam Sung Kim, "Chameleon: Versatile and Practical Near-DRAM Acceleration Architecture for Large Memory Systems," in *MICRO*, 2016.
- [23] Hanhwi Jang, Joonsung Kim, Jae-Eon Jo, Jaewon Lee, Jangwoo Kim, "MnnFast: a fast and scalable system architecture for memory-augmented neural networks," in *ISCA*, 2019.
- [24] Hardik Sharma, Jongse Park, Naveen Suda, Liangzhen Lai, Benson Chau, Vikas Chandra, Hadi Esmaeilzadeh, "Bit Fusion: Bit-Level Dynamically Composable Architecture for Accelerating Deep Neural Network," in *ISCA*, 2018.
- [25] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishu Aradhye, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, Rohan Anil, Zakaria Haque, Lichan Hong, Vihan Jain, Xiaobing Liu, Hemal Shah, "Wide & Deep Learning for Recommender Systems," in *RecSys*, 2016, pp. 7–10.
- [26] Hsing-Min Chen, Akhil Arunkumar, Carole-Jean Wu, Trevor Mudge, Chaitali Chakrabarti, "E-ecc: Low power erasure and error correction schemes for increasing reliability of commodity dram systems," in *MEMSYS*, 2015.
- [27] Hsing-Min Chen, Carole-Jean Wu, Trevor Mudge, Chaitali Chakrabarti, "RATT-ECC: Rate adaptive two-tiered error correction codes for reliable 3D die-stacked memory," in *ACM Transactions on Architecture and Code Optimization*, vol. 13, no. 3, 2016.
- [28] Hsing-Min Chen, Supreet Jeloka, Akhil Arunkumar, David Blaauw, Carole-Jean Wu and Trevor Mudge and Chaitali Chakrabarti, "Using low cost erasure and error correction schemes to improve reliability of commodity dram systems," vol. 65, no. 12, 2016.
- [29] Hui Feng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, Xiuqiang He, Zhenhua Dong, "DeepFM: An End-to-End Wide & Deep Learning Framework for CTR Prediction," in *IJCAI*, 2017.
- [30] Intel Memory Latency Checker (MLC), <https://software.intel.com/en-us/articles/intel-memory-latency-checker>.
- [31] Jaekyu Lee, Hyesoon Kim, and Richard Vuduc, "When Prefetching Works, When It Doesn't, and Why," in *ACM TACO*, vol. 9, no. 1, 2012.
- [32] Jiawen Liu, Hengyu Zhao, Matheus Almeida Ogleari, Dong Li, Jishen Zhao, "Processing-in-Memory for Energy-efficient Neural Network Training: A Heterogeneous Approach," in *MICRO*, 2018, pp. 655–668.
- [33] Jiecao Yu, Andrew Lukefahr, David J. Palfaman, Ganesh S. Dasika, Reetuparna Das, Scott A. Mahlke, "Scalpel: Customizing DNN Pruning to the Underlying Hardware Parallelism," in *ISCA*, 2017.
- [34] Jiquan Ngiam, Zhenghao Chen, Daniel Chia, Pang W. Koh, Quoc V. Le, Andrew Y. Ng, "Tiled convolutional neural networks," in *NIPS*, 2010, pp. 1279–1287.
- [35] John E. Stone, David Gohara, Guochun Shi, "OpenCL: A Parallel Programming Standard for Heterogeneous Computing Systems," in *IEEE Computing in Science and Engineering*, vol. 12, no. 3, 2010.
- [36] Jorge Albericio, Patrick Judd, Tayler Hetherington, Tor Aamodt, Natalie Enright Jerger, Andreas Moshovos, "Cnvlutin: Ineffectual-neuron-free Deep Neural Network Computing," in *ISCA*, 2016.
- [37] Joseph Izraelevitz, Jian Yang, Lu Zhang, Juno Kim, Xiao Liu, Amir-saman Memaripour, Yun Joon Soh, Zixuan Wang, Yi Xu, Subramanya R. Dulloor, Jishen Zhao, Steven Swanson, "Basic Performance Measurements of the Intel Optane DC Persistent Memory Module," in *arXiv preprint arXiv:1903.05714*, 2018.
- [38] Junwhan Ahn, Sungjoo Yoo, Onur Mutlu, Kiyoun Choi, "PIM-Enabled Instructions: A Low-Overhead, Locality-Aware Processing-in-Memory Architecture," in *ISCA*, 2015, pp. 336–348.
- [39] Junwhan Ahn, Sungpack Hong, Sungjoo Yoo, Onur Mutlu, Kiyoun Choi, "A scalable processing-in-memory accelerator for parallel graph processing," pp. 105–117, 2015.
- [40] Kartik Hegde, Jiyong Yu, Rohit Agrawal, Mengjia Yan, Michael Pellauer, Christopher W. Fletcher, "UCNN: Exploiting Computational Reuse in Deep Neural Networks via Weight Repetition," in *ISCA*, 2018.
- [41] Ke Chen, Sheng Li, Naveen Muralimanohar, Jung Ho Ahn, Jay B. Brockman, Norman P. Jouppi, "Cacti-3dd: Architecture-level modeling for 3d die-stacked dram main memory," in *DATE*, 2012.
- [42] Kevin Hsieh, Eiman Ebrahimi, Gwangsun Kim, Niladri Chatterjee, Mike O'Connor, Nandita Vijaykumar, Onur Mutlu, Stephen W. Keckler, "Transparent Offloading and Mapping (TOM): Enabling Programmer-Transparent Near-Data Processing in GPU Systems," in *ISCA*, 2016, pp. 204–216.
- [43] Kevin Hsieh, Samira Khan, Nandita Vijaykumar, Kevin K. Chang, Amirali Boroumand, Saugata Ghose, Onur Mutlu, "Accelerating pointer chasing in 3d-stacked memory: Challenges, mechanisms, evaluation," in *ICCD*, 2016, pp. 25–32.
- [44] Kim Hazelwood, Sarah Bird, David Brooks, Soumith Chintala, Utku Diril, Dmytro Dzhulgakov, Mohamed Fawzy, Bill Jia, Yangqing Jia, Aditya Kalro, James Law, Kevin Lee, Jason Lu, Pieter Noordhuis, Misha Smelyanskiy, Liang Xiong, Xiaodong Wang, "Applied machine learning at Facebook: a datacenter infrastructure perspective," in *HPCA*, 2018, pp. 620–629.
- [45] Lifeng Nai, Ramyad Hadidi, Jaewoong Sim, Hyeonjong Kim, Pranith Kumar, Hyesoon Kim, "GraphPIM: Enabling Instruction-Level PIM Offloading in Graph Computing Frameworks," in *HPCA*, 2017.
- [46] M. Gao, J. Pu, X. Yang, M. Horowitz, and C. Kozyrakis, "Tetris: Scalable and efficient neural network acceleration with 3d memory," in *ASPLOS*, 2017, pp. 751–764.
- [47] Marc Riera, Jose Maria Arnaiz, Antonio Gonzalez, "Computation Reuse in DNNs by Exploiting Input Similarity," in *ISCA*, 2018.
- [48] Mario Drumond, Alexandros Daglis, Nooshin Sadat Mirzadeh, Dmitrii Ustiugov, Javier Picorel, Babak Falsafi, Boris Grot, Dionisios N Pnevmatikatos, "The mondrian data engine," in *ISCA*, 2017, pp. 639–651.
- [49] Maxim Naumov, Dheevatsa Mudigere, Hao-Jun Michael Shi, Jianyu Huang, Narayanan Sundaraman, Jongsoo Park, Xiaodong Wang, Udit Gupta, Carole-Jean Wu, Alisson G. Azzolini, Dmytro Dzhulgakov, Andrey Malleevich, Ilia Cherniavskii, Yinghai Lu, Raghuraman Krishnamoorthi, Ansha Yu, Volodymyr Kondratenko, Stephanie Pereira, Xianjie Chen, Wenlin Chen, Vijay Rao, Bill Jia, Liang Xiong, Misha Smelyanskiy, "Deep Learning Recommendation Model for Personalization and Recommendation Systems," in *arXiv preprint arXiv:1906.00091*, 2019. [Online]. Available: <https://arxiv.org/abs/1906.00091>
- [50] Mel Gorman, "Understanding the Linux virtual memory manager," 2004.
- [51] Micron, "MT40A2G4, MT40A1G8, MT40A512M16, 8Gb: x4, x8, x16 DDR4 SDRAM Features."
- [52] Miguel Campo, Cheng-Kang Hsieh, Matt Nickens, J.J. Espinoza, Abhinav Taliyan, Julie Rieger, Jean Ho, and Bettina Sherrick, "Competitive Analysis System for Theatrical Movie Releases Based on Movie Trailer Deep Video Representation," in *Arxiv*, 2018. [Online]. Available: <https://arxiv.org/abs/1807.04465>
- [53] Mingcong Song, Jiaqi Zhang, Huixiang Chen, Tao Li, "Towards Efficient Microarchitectural Design for Accelerating Unsupervised GAN-Based Deep Learning," in *HPCA*, 2018.
- [54] Mingcong Song, Jiechen Zhao, Yang Hu, Jiaqi Zhang, Tao Li, "Prediction based Execution on Deep Neural Networks," in *ISCA*, 2018.
- [55] Mingcong Song, Kan Zhong, Jiaqi Zhang, Yang Hu, Duo Liu, Weigong Zhang, Jing Wang, Tao Li, "In-Situ AI: Towards Autonomous and Incremental Deep Learning for IoT Systems," in *HPCA*, 2018.
- [56] Mingyu Gao, Grant Ayers, Christos Kozyrakis, "Practical Near-Data Processing for In-Memory Analytics Frameworks," in *PACT*, 2015, pp. 113–124.
- [57] Minsoo Rhu, Mike O'Connor, Niladri Chatterjee, Jeff Pool, Youngeun Kwon, Stephen W. Keckler, "Compressing DMA Engine: Leveraging Activation Sparsity for Training Deep Neural Networks," in *HPCA*, 2018.
- [58] Naveen Muralimanohar, Rajeev Balasubramanian, and Norman P. Jouppi, "Cacti 6.0: A tool to model large caches," in *HP laboratories*, 2009, pp. 22–31.
- [59] Norman P. Jouppi, Andrew B. Kahng, Naveen Muralimanohar, Vaishnav Srinivas, "Cacti-io: Cacti with off-chip power-area-timing models," in *VLSI*, 2015, pp. 1254–1267.
- [60] Norman P. Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, Rick Boyle, Pierre-luc Cantin, Clifford Chao, Chris Clark, Jeremy Coriell, Mike Daley, Matt Dau, Jeffrey Dean, Ben Gelb, Tara Vazir Ghaemmaghami, Rajendra Gottipati, William Gulland, Robert Hagmann, C. Richard Ho, Doug Hogberg, John Hu, Robert Hundt, Dan Hurt, Julian Ibarz, Aaron Jaffey, Alek Jaworski, Alexander Kaplan, Harshit Khaitan, Andy Koch, Naveen Kumar, Steve Lacy, James Laudon, James Law, Diemthu Le, Chris Leary, Zhuyuan Liu, Kyle Lucke, Alan

- Lundin, Gordon MacKean, Adriana Maggiore, Maire Mahony, Kieran Miller, Rahul Nagarajan, Ravi Narayanaswami, Ray Ni, Kathy Nix, Thomas Norrie, Mark Omerick, Narayana Penukonda, Andy Phelps, Jonathan Ross, Matt Ross, Amir Salek, Emad Samadiani, Chris Severn, Gregory Sizikov, Matthew Snelham, Jed Souter, Dan Steinberg, Andy Swing, Mercedes Tan, Gregory Thorson, Bo Tian, Horia Toma, Erick Tuttle, Vijay Vasudevan, Richard Walter, Walter Wang, Eric Wilcox, Doe Hyun Yoon, "In-datacenter performance analysis of a tensor processing unit," in *ISCA*, 2017, pp. 1–12.
- [61] Onur Mutlu, Thomas Moscibroda, "Stall-Time Fair Memory Access Scheduling for Chip Multiprocessors," in *MICRO*, 2007, pp. 146–160.
- [62] Patrick J. Meaney, Lawrence D. Curley, Glenn D. Gilda, Mark R. Hodges, Daniel J. Buerkle, Robert D. Siegl, Roger K. Dong, "The IBM z13 Memory Subsystem for Big Data," in *IBM Journal of Research and Development*, 2015.
- [63] Paul Covington, Jay Adams, Emre Sargin, "Deep Neural Networks for YouTube Recommendations," in *RecSys*, 2016, pp. 191–198.
- [64] Peter Mattson, Christine Cheng, Cody Coleman, Greg Diamos, Paulius Micikevicius, David Patterson, Hanlin Tang, Gu-Yeon Wei, Peter Bailis, Victor Bittorf, David Brooks, Dehao Chen, Debojyoti Dutta, Udit Gupta, Kim Hazelwood, Andrew Hock, Xinyuan Huang, Atsushi Ike, Bill Jia, Daniel Kang, David Kanter, Naveen Kumar, Jeffery Liao, Guokai Ma, Deepak Narayanan, Tayo Oguntebi, Gennady Pekhimenko, Lillian Pentecost, Vijay Janapa Reddi, Taylor Robie, Tom St. John, Tsuguchika Tabaru, Carole-Jean Wu, Lingjie Xu, Masafumi Yamazaki, Cliff Young, Matei Zaharia, "MLPerf Training Benchmark," *arXiv preprint arXiv:1910.01500*, 2019.
- [65] Peter Mattson, Paulius Micikevicius, Vijay Janapa Reddi, David Patterson, Christine Cheng, Guenther Schmuelling, Cody Coleman, Hanlin Tang, Greg Diamos, Gu-Yeon Wei, David Kanter, Carole-Jean Wu, "MLPerf: An Industry Standard Benchmark Suite for Machine Learning Performance," *IEEE Micro*, vol. 40, no. 2, pp. 8–16, 2020.
- [66] Peter Pessl, Daniel Gruss, Cl  mentine Maurice, Michael Schwarz, Stefan Mangard, "Drama: Exploiting dram addressing for cross-cpu attack," in *USENIX Security Symposium*, vol. pp.565-581, 2016.
- [67] Qi Guo, N. Alachiotis, Berkin Akin, F. Sadi, G. Xu, Tze-Meng Low, Lawrence Pileggi, James C. Hoe, Franz Franchetti, "3d-stacked memory-side acceleration: Accelerator and system design," in *WoNDP*, 2014.
- [68] Reza Yazdani, Marc Riera, Jose-Maria Arna  , Antonio Gonzalez, "The Dark Side of DNN Pruning," in *ISCA*, 2018.
- [69] Robert Adolf, Saketh Rama, Brandon Reagen, Gu-Yeon Wei, David Brooks, "Fathom: Reference workloads for modern deep learning methods," in *IISWC*, 2016, pp. 1–10.
- [70] Samuel Williams, Andrew Waterman, and David Patterson, "Roofline: An Insightful Visual Performance Model for Floating-Point Programs and Multicore Architectures," in *Communications of the ACM*, 2009.
- [71] Shijin Zhang, Zidong Du, Lei Zhang, Huiying Lan, Shaoli Liu, Ling Li, Qi Guo, Tianshi Chen, Yunji Chen, "Cambricon-x: An accelerator for sparse neural networks," in *MICRO*, 2016.
- [72] Song Han, Xingyu Liu, Huizi Mao, Jing Pu, Ardavan Pedram, Mark A Horowitz, William J Dally, "EIE: efficient inference engine on compressed deep neural network," in *ISCA*, 2016, pp. 243–254.
- [73] Swagath Venkataramani, Ashish Ranjan, Subarno Banerjee, Dipankar Das, Sasikanth Avancha, Ashok Jagannathan, Ajaya Durg, Dheemanth Nagaraj, Bharat Kaul, Pradeep Dubey, Anand Raghunathan, "ScaleDeep: A Scalable Compute Architecture for Learning and Evaluating Deep Networks," in *ISCA*, 2017.
- [74] Udit Gupta, Brandon Reagen, Lillian Pentecost, Marco Donato, Thierry Tambe, Alexander M Rush, Gu-Yeon Wei, David Brooks, "MASR: A Modular Accelerator for Sparse RNNs," in *PACT*, 2019, pp. 1–14.
- [75] Udit Gupta, Carole-Jean Wu, Xiaodong Wang, Maxim Naumov, Brandon Reagen, David Brooks, Bradford Cottel, Kim Hazelwood, Bill Jia, Hsien-Hsin S. Lee, Andrey Malevich, Dheevatsa Mudigere, Mikhail Smelyanskiy, Liang Xiong, Xuan Zhang, "The Architectural Implications of Facebook's DNN-based Personalized Recommendation," in *HPCA*, 2020.
- [76] Vahide Aklaghi, Amir Yazdanbakhsh, Kambiz Samadi, Hadi Esmaeilzadeh, Rajesh K. Gupta, "SnaPEA: Predictive Early Activation for Reducing Computation in Deep Convolutional Neural Networks," in *ISCA*, 2018.
- [77] Vijay Janapa Reddi, Christine Cheng, David Kanter, Peter Mattson, Guenther Schmuelling, Carole-Jean Wu, Brian Anderson, Maximilien Breughe, Mark Charlebois, William Chou, Ramesh Chukka, Cody Coleman, Sam Davis, Pan Deng, Greg Diamos, Jared Duke, Dave Fick, J. Scott Gardner, Itay Hubara, Sachin Idgunji, Thomas B. Jablin, Jeff Jiao, Tom St. John, Pankaj Kanwar, David Lee, Jeffery Liao, Anton Lokhmotov, Francisco Massa, Peng Meng, Paulius Micikevicius, Colin Osborne, Gennady Pekhimenko, Arun Tejusve Raghunath Rajan, Dilip Sequeira, Ashish Sirasao, Fei Sun, Hanlin Tang, Michael Thomson, Frank Wei, Ephrem Wu, Lingjie Xu, Koichi Yamada, Bing Yu, George Yuan, Aaron Zhong, Peizhao Zhang, Yuchen Zhou, "Mlperf inference benchmark," in *arXiv preprint arXiv:1911.02549*, 2019.
- [78] Xiao Zhang, Sandhya Dwarkadas, Kai Shen, "Towards practical page coloring-based multicore cache management," in *EuroSys*, 2009, pp. 89–102.
- [79] Yongming Shen, Michael Ferdman, Peter Milder, "Maximizing CNN Accelerator Efficiency Through Resource Partitioning," in *ISCA*, 2017.
- [80] Yoongu Kim, Weikun Yang and Onur Mutlu, "Ramulator: A fast and extensible DRAM simulator," in *IEEE Computer architecture letters*, vol. 15, no. 1. IEEE, 2015, pp. 45–49.
- [81] Youngeun Kwon, Yunjae Lee, Minsoo Rhu, "TensorDIMM: A Practical Near-Memory Processing Architecture for Embeddings and Tensor Operations in Deep Learning," in *MICRO*, 2019, pp. 740–753.
- [82] Yu-Hsin Chen, Tushar Krishna, Joel S. Emer and Vivienne Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," in *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, 2017, pp. 127–138.
- [83] Zhe Zhao, Lichan Hong, Li Wei, Jilin Chen, Aniruddh Nath, Shawn Andrews, Aditee Kumthekar, Maheswaran Sathiamoorthy, Xinyang Yi, Ed Chi, "Recommending What Video to Watch Next: A Multitask Ranking System," in *RecSys*, 2019, pp. 43–51.